

## RLM в действии · trace выполнения на реальном вопросе

ВОПРОС: «Сколько в датасете 500 записей пар вопросов, где ОБА про писателей?»

1

LLM получает только метаданные

LLM видит: `len(context) = 32 000` символов · тип: «список TREC-вопросов» · prefix: «#1 Столица Японии – LOC, #2 Кто Толстой – HUM, ...»  
LLM думает: «Сначала найду все HUM-записи через простой фильтр, потом для каждой проверю – про писателя или нет»

2

Пишет фильтрующий код → REPL выполняет

```
records = context.split('\n')
hum = [r for r in records if r.endswith('HUM')]
print(f"{len(hum)} HUM-записей")
```

OUTPUT:

87 HUM-записей

3

LLM решает: пора рекурсию! Каждую HUM-запись надо проверить «про писателя или нет»

```
writers = []
for r in hum:
    answer = sub_RLM(
        query="Этот вопрос про писателя? Да/Нет",
        context=r
    )
    if answer == "Да":
        writers.append(r)
print(f"{len(writers)} писателей")
```

# 87 итераций  
# ← РЕКУРСИЯ – новый RLM на этой одной записи  
# ~50 символов

запускается 87 sub\_RLM-вызовов

DEPTH = 1 · 87 параллельных мини-RLM, каждый с чистым контекстом, видит ОДНУ запись

sub\_RLM #1  
«#2 Кто написал Войну и Мир?»  
→ ✓ Да (Толстой = писатель)

sub\_RLM #2  
«#5 Кто изобрёл телефон?»  
→ ✗ Нет (Белл = изобретатель)

sub\_RLM #3  
«#7 Кто написал Гарри Поттер?»  
→ ✓ Да (Роулинг = писатель)

... ещё 84 sub\_RLM вызова ...

Каждый sub\_RLM: контекст 50 символов · чистая голова · работает 1–2 секунды · стоит копейки  
Итого: 15 ответов «Да», 72 ответа «Нет»

4

Результаты возвращаются как переменная в главный REPL

```
writers = [«#2 Кто Толстой?», «#7 Кто Роулинг?», «#23 Кто Достоевский?» ...] # 15 записей
приходит в основной REPL как обычный массив строк. История 87 sub-вызовов отбрасывается.
```

5

Считает финальный ответ и записывает в Final

```
n = len(writers) # 15
pairs = n * (n - 1) // 2 # 15 * 14 / 2 = 105
Final = f"{pairs} пар"
```

RETURN

«105 пар» ✓

💡 Главная LLM никогда не «видела» сами 32 000 токенов. Она работала только с метаданными и outputs от Python.  
87 раз нырнули в данные через sub\_RLM – каждый раз с маленьким куском в чистой голове. Результат: точное число, дёшево, без шума.